

## R3.04 : Design patterns

### Patrons de conception (2)

Arnaud Lanoix Brauer

`Arnaud.Lanoix@univ-nantes.fr`



**IUT Nantes**

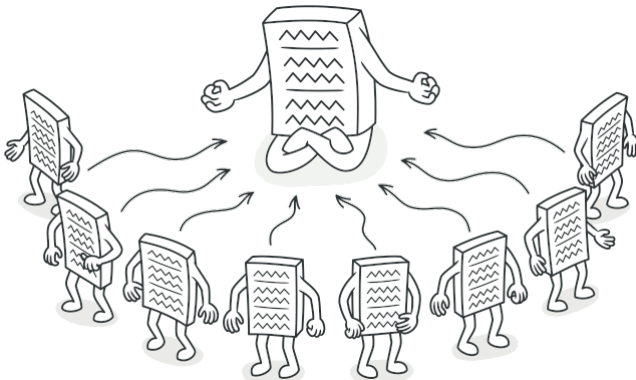
Pôle Sciences et technologie

**Nantes Université**

Département informatique

- 1 Patrons de conception créateur
- 2 Patrons de conception comportementaux

# Patron Singleton – Singleton pattern



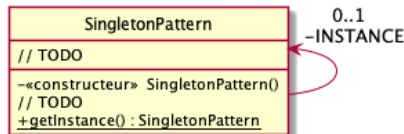
# Patron Singleton – Singleton pattern

## Problème

Garantir qu'une classe n'a **qu'une SEULE et UNIQUE** instance

## Solution

La classe à créer va fournir un moyen d'accéder à l'instance unique



- Singleton est utilisé lorsqu'on a besoin **d'exactly** un objet pour coordonner des opérations dans un système
- Permet d'éviter des **instanciations multiples**, par exemple
  - ▶ accès à une BDD
  - ▶ flux en écriture vers un fichier
  - ▶ connexion à serveur

Utilise le design pattern **Factory**

# Implémentation(s) de singleton en Kotlin

## Implémentation "classique"

```
class SingletonPattern
    private constructor() {

        companion object {

            private val INSTANCE
                = SingletonPattern()

            fun getInstance()
                : SingletonPattern {
                return INSTANCE
            }
        }
    }
}
```

## Initialisation "paresseuse"

```
class SingletonPattern
    private constructor() {

        companion object {

            private var INSTANCE :
                SingletonPattern?

            fun getInstance() :
                SingletonPattern {
                if (INSTANCE == null)
                    INSTANCE =
                        SingletonPattern()
                return INSTANCE!!
            }
        }
    }
}
```

Le constructeur est privé.

La variable statique **INSTANCE** n'est initialisée qu'une fois.

# Implémentation(s) de singleton en Kotlin

## Implémentation "classique"

```
class SingletonPattern
    private constructor() {

        companion object {

            private val INSTANCE
                = SingletonPattern()

            fun getInstance()
                : SingletonPattern {
                return INSTANCE
            }
        }
    }
}
```

Le constructeur est privé.

La variable statique `INSTANCE` n'est initialisée qu'une fois.

## Initialisation "paresseuse"

```
class SingletonPattern
    private constructor() {

        companion object {

            private var INSTANCE :
                SingletonPattern?

            fun getInstance() :
                SingletonPattern {
                if (INSTANCE == null)
                    INSTANCE =
                        SingletonPattern()
                return INSTANCE!!
            }
        }
    }
}
```

# Implémentation(s) simplifiée(s) de Singleton en Kotlin

## en utilisant "by lazy"

```
class SingletonPattern
private constructor() {

    companion object {

        private val INSTANCE
            by lazy {
                SingletonPattern()
            }

        fun getInstance() =
            INSTANCE
    }
}
```

## en utilisant "object"

Kotlin propose une construction spécifique de Singleton : `object`

```
object SingletonPattern2 {
    // TODO
}
```

## ATTENTION

Plus compliquer à assurer dans le cas de programmation multi-threadée.

# Implémentation(s) simplifiée(s) de Singleton en Kotlin

## en utilisant "by lazy"

```
class SingletonPattern
private constructor() {

    companion object {

        private val INSTANCE
            by lazy {
                SingletonPattern()
            }

        fun getInstance() =
            INSTANCE
    }
}
```

## en utilisant "object"

Kotlin propose une construction  
spécifique de Singleton : `object`

```
object SingletonPattern2 {
    // TODO
}
```

## ATTENTION

Plus compliquer à assurer dans le cas de programmation multi-threadée.

# Implémentation(s) simplifiée(s) de Singleton en Kotlin

## en utilisant "by lazy"

```
class SingletonPattern
private constructor() {

    companion object {

        private val INSTANCE
            by lazy {
                SingletonPattern()
            }

        fun getInstance() =
            INSTANCE
    }
}
```

## en utilisant "object"

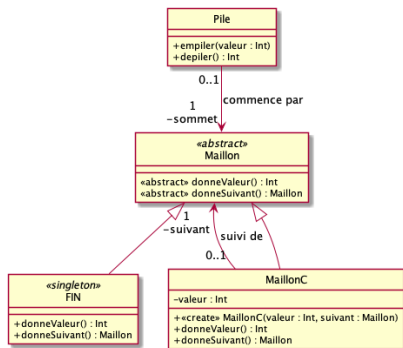
Kotlin propose une construction  
spécifique de Singleton : `object`

```
object SingletonPattern2 {
    // TODO
}
```

## ATTENTION

Plus compliquer à assurer dans le cas de programmation multi-threadée.

# Exemple d'utilisation d'un singleton (1)



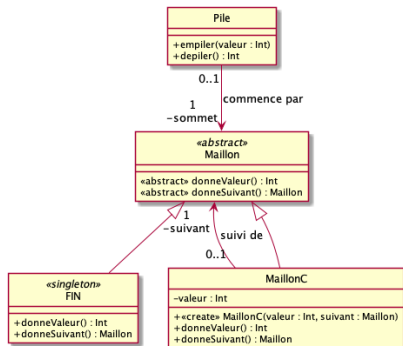
```
abstract class Maillon {
    abstract fun donneValeur() : Int
    abstract fun donneSuivant() : Maillon
}

class MaillonC(
    private val valeur : Int,
    private var suivant : Maillon = FIN
) : Maillon() {

    override fun donneValeur() = valeur
    override fun donneSuivant() = suivant
}

object FIN : Maillon() {
    override fun donneValeur() =
        throw Exception("Fin de pile")
    override fun donneSuivant() =
        throw Exception("Fin de pile")
}
```

## Exemple d'utilisation d'un singleton (2)

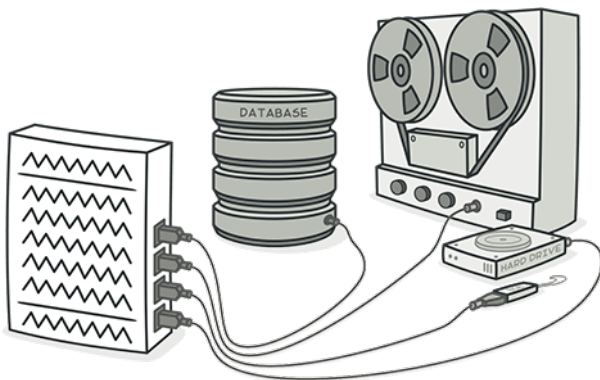


```
class Pile {
    private var sommet : Maillon = FIN

    fun empiler(valeur : Int) {
        sommet = MaillonC(valeur, sommet)
    }

    fun depiler() : Int {
        if (sommet == FIN)
            throw Exception("Pile vide")
        val valeur = sommet.donneValeur()
        sommet = sommet.donneSuivant()
        return valeur
    }
}
```

# Patron Objet d'accès aux données – DAO pattern



# Patron Objet d'accès aux données – DAO pattern

## Problème

Les **objets métiers** instanciés sont liés à des **données persistentes** (fichiers, BDD)

## Solution

Le patron DAO (Data Access Object) propose de séparer les **classes (de données) métiers**, de **classes "techniques"** réalisant la liaison avec le stockage persistant.

- Les classes métiers ne sont modifiées que si les règles métiers changent
- On peut changer la méthode d'accès aux données sans impacter les classes métiers

## Pour aller plus loin : Repository

La différence est que **DAO** focuse sur la persistance des données alors que **Repository** focuse sur l'abstraction des données.

# Patron Objet d'accès aux données – DAO pattern

## Problème

Les **objets métiers** instanciés sont liés à des **données persistentes** (fichiers, BDD)

## Solution

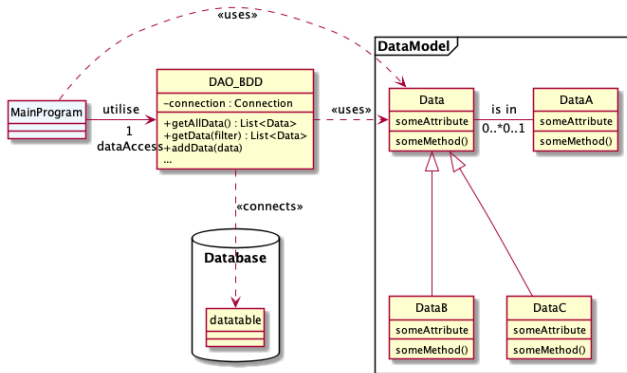
Le patron DAO (Data Access Object) propose de séparer les **classes (de données) métiers**, de **classes "techniques"** réalisant la liaison avec le stockage persistant.

- Les classes métiers ne sont modifiées que si les règles métiers changent
- On peut changer la méthode d'accès aux données sans impacter les classes métiers

## Pour aller plus loin : Repository

La différence est que **DAO** focuse sur la persistance des données alors que **Repository** focuse sur l'abstraction des données.

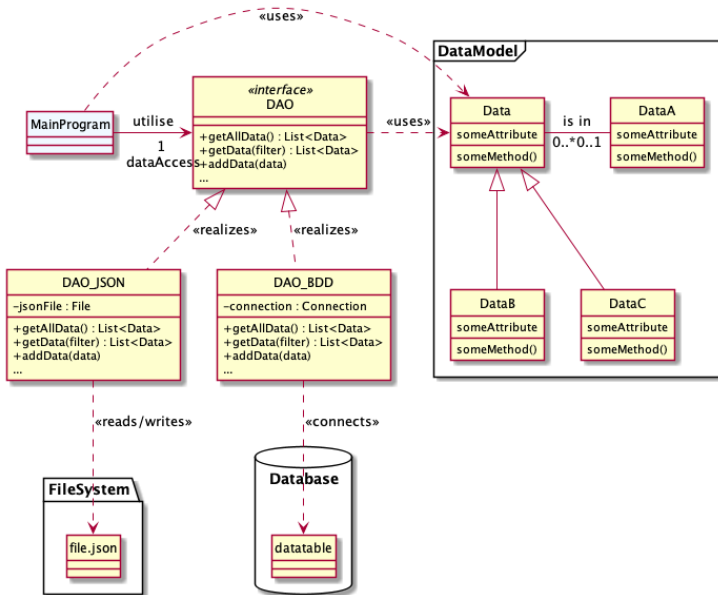
# Patron DAO : schéma UML



## CRUD

La classe DAO implémente tout ou partie des **opérations CRUD** sur la BDD, en fonction du besoin client.

# Patron DAO : schéma UML (2)



# Exemple de DAO / JSON

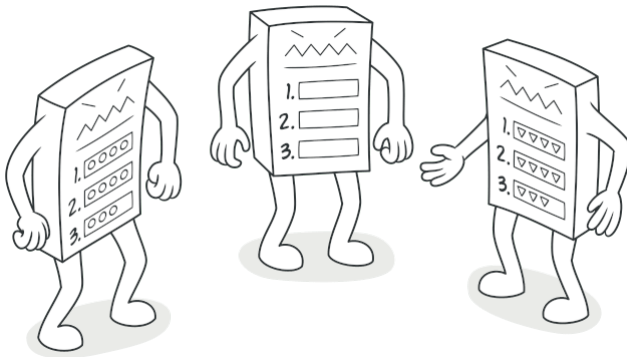
```
[  
  { "nom" : "Lego",  
    "prix": 10.0,  
    "quantite" : 30 },  
  { "nom" : "Playmobil",  
    "prix": 20.0,  
    "quantite" : 50 },  
  ...  
]
```

```
@Serializable  
data class Produit(  
    val nom : String,  
    var prix : Double,  
    var quantite : Int  
)
```

```
class ProduitDAO (val json : File) {  
  
    fun donneTousLesProduits() : List<Produit> {  
        return Json  
            .decodeFromStream<List<Produit>>(json.inputStream())  
    }  
  
    fun donneProduit(nom : String) : Produit {  
        return Json  
            .decodeFromStream<List<Produit>>(json.inputStream())  
            .filter {it.nom == nom} .get(0)  
    }  
}
```

- 1 Patrons de conception créateur
- 2 Patrons de conception comportementaux

# Patron de méthode – Template method pattern



# Patron de méthode – Template method pattern

## Problème

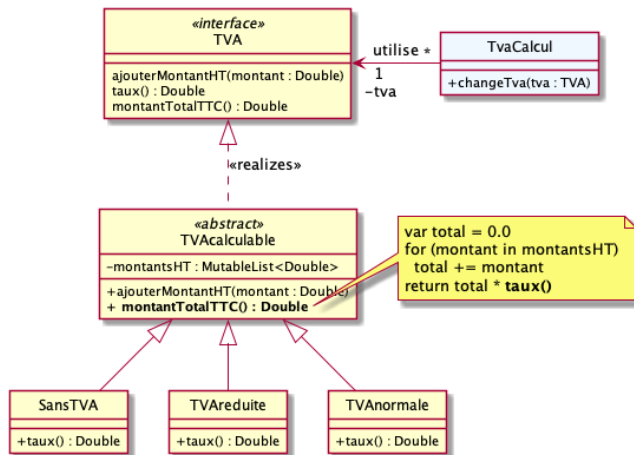
Comment **généraliser** un algorithme, dont uniquement certaines étapes sont **spécifiques** ? La spécificité dépend de la **sous-classe** sur laquelle s'applique l'algorithme

## Solution

Le patron de méthode propose de

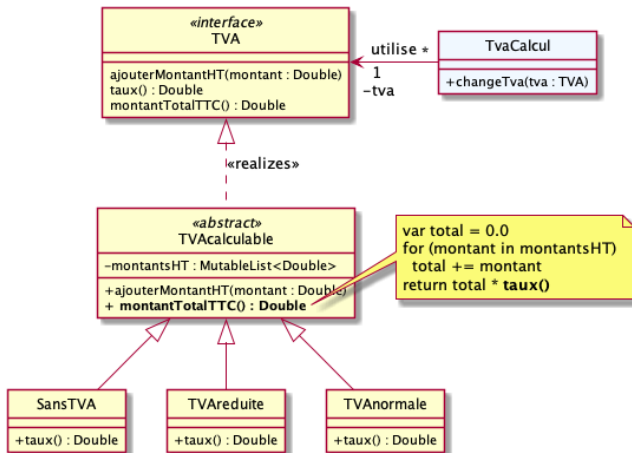
- 1 définir les **parties spécifiques** comme des méthodes d'une interface
  - 2 implémenter le **squelette de l'algorithme** en utilisant les méthodes définies par l'interface
- permet de factoriser du code qui serait redondant
  - l'**algorithme "à trou"** est défini dans une classe abstraite parente (ou dans une autre classe, en combinaison avec **Strategy**)

# Exemple de Patron de méthode



Ici la méthode `montantTotalTTC()` n'est pas **open** pour éviter toute modification de l'algorithme (par héritage/redefinition)

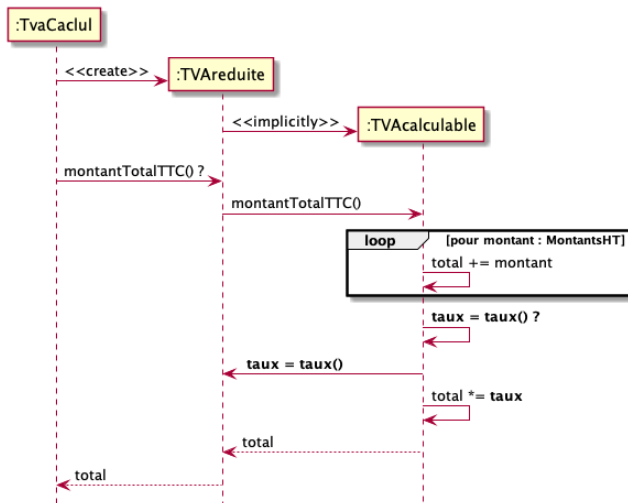
# Exemple de Patron de méthode



Ici la méthode `montantTotalTTC()` n'est pas **open** pour éviter toute modification de l'algorithme (par héritage/redefinition)

## Exemple de Patron de méthode (2)

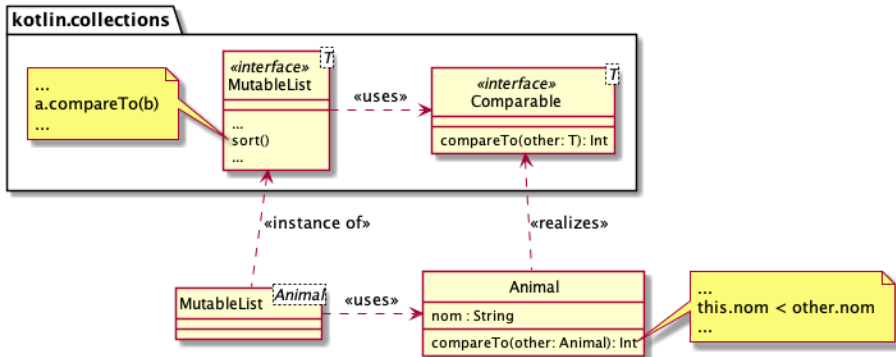
Un diagramme de séquence UML permet d'illustrer la méthode "à trou".



# La méthode Kotlin `sort()`

La méthode `<T : Comparable<T>> MutableList<T>.sort()` utilise le **patron de méthode** :

Les éléments `T` doivent implémenter `Comparable<T>` pour pouvoir être triés



# Patron Stratégie – Strategy Patterns



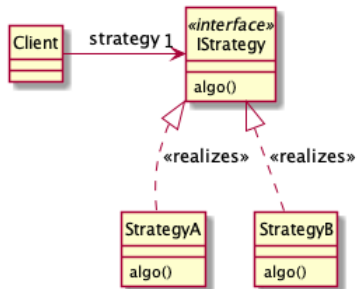
# Patron Stratégie – Strategy Patterns

## Problème

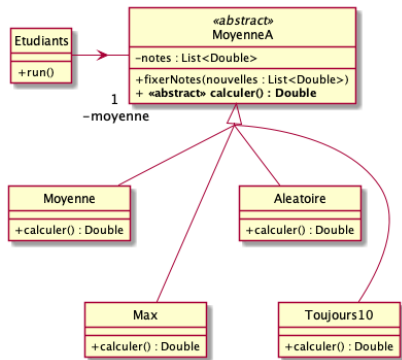
On a plusieurs algorithmes **similaires** ; comment **changer** facilement l'algorithme utilisé ?

## Solution

- Définir la famille d'algorithmes grâce à une **interface**/classe abstraite
- Définir chaque algo par une classe **implémentant** l'interface
- Le client utilise l'un des algorithmes à un moment donné via un **attribut** référençant la stratégie
- On peut facilement changer la classe concrète utilisée et donc l'algorithme



# Exemple de Stratégie



```
abstract class MoyenneA {
    protected lateinit var notes : List<Double>

    fun fixerNotes(nouvelles : List<Double>) {
        notes = nouvelles
    }

    abstract fun calculer() : Double
}
```

```
class Toujours10() : MoyenneA() {
    override fun calculer() = 10.0
}
```

```
class Aleatoire() : MoyenneA() {
    override fun calculer() = notes.random()
}
```

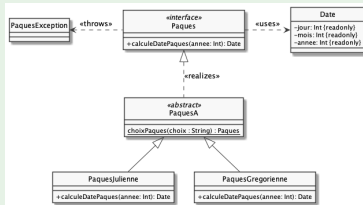
```
class Max() : MoyenneA() {
    override fun calculer() = notes.maxOf {it}
}
```

```
class Moyenne() : MoyenneA() {
    override fun calculer() = notes.average()
}
```

```
when (enseignants.random()) {
    "AL" -> moyenne = Toujours10()
    "JFB" -> moyenne = Aleatoire()
    "JFR" -> moyenne = Max()
    "JMM" -> moyenne = Moyenne()
}
for (notes in etudiants) {
    moyenne.fixerNotes(notes)
    println(" ${moyenne.calculer()} ")
}
```

# Autres exemples de stratégies

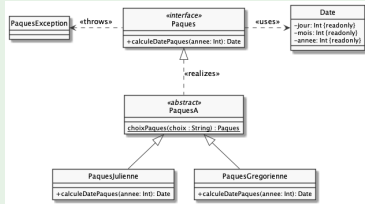
## Calcul de Pâques



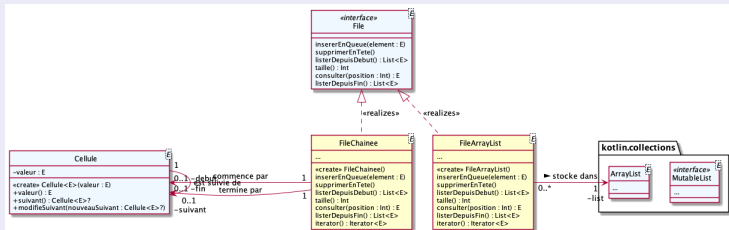
File(s)

# Autres exemples de stratégies

## Calcul de Pâques



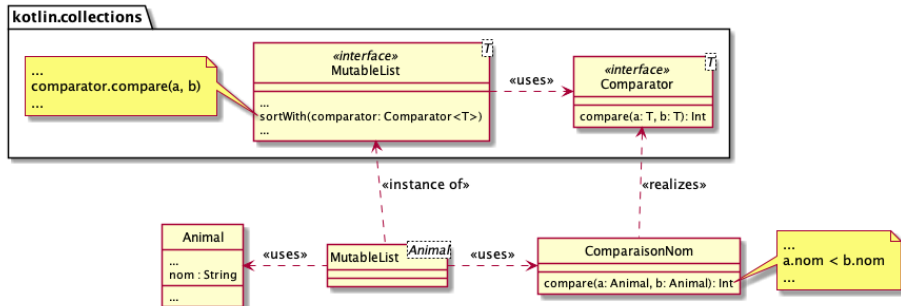
## File(s)



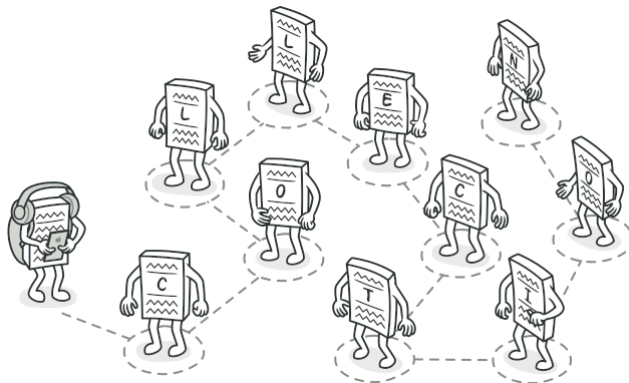
# L'interface Kotlin `Comparator<T>`

`Comparator<T>` propose de définir une **stratégie d'ordonnancement** des éléments `T`.

Cette stratégie est ensuite utilisée, par exemple pour **trier une liste** :  
`MutableList<T>.sortWith(comparator : Comparator<T>)`



# Patron Itérateur – Iterator Pattern



# Patron Itérateur – Iterator Pattern

## Problème

Comment parcourir **séquentiellement** un objet conteneur (liste, arbre, etc.) sans en **dévoiler la structure interne** (potentiellement complexe) ?

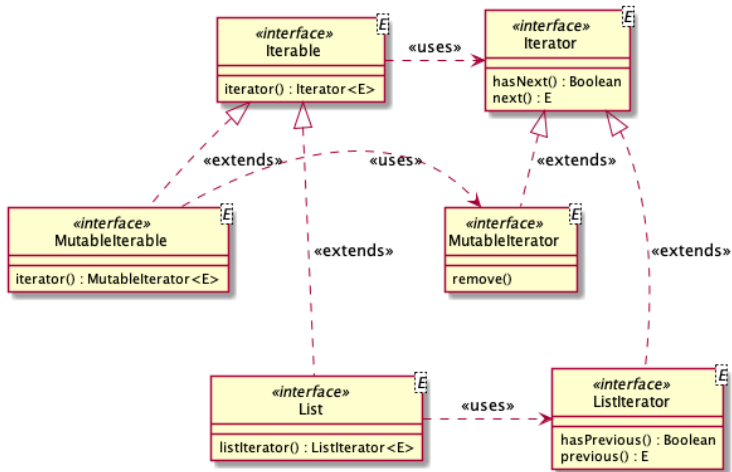
## Solution

Le conteneur propose un **itérateur** qui va permettre un parcours séquentiel, c-à-d

- d'accéder à l'élément courant
  - de passer à l'élément "suivant"
  - de déterminer si on a tout parcouru
- 
- Un itérateur permet l'accès séquentiel de structures **non-indexées/non-ordonnées** : ensembles, arbres, etc.
  - Certains itérateurs proposent de **modifier** le conteneur pendant le parcours, d'itérer dans /alerti'autre sens

# L'interface `Iterable<E>` en Kotlin

En Kotlin, toutes les collections standard de `kotlin.collections` héritent de `Iterable<E>`



# Utilisation d'un itérateur en Kotlin

```
val teachers = mutableListOf<String>("Arnaud", "JFB",  
                                     "JFR", "JM", "Olivier")  
  
val ite = teachers.iterator()  
while(ite.hasNext()) {  
    val teacher = ite.next()  
    print("$teacher ")  
}
```

est équivalent à

```
for (teacher in teachers) {  
    print("$teacher ")  
}
```

On peut modifier la collection à travers l'itérateur :

```
while(ite.hasNext()) {  
    val teacher = ite.next()  
    if (teacher == "JFB")  
        ite.remove()  
}
```